

Chapter 28: Performance

1. Overview

Angular performance work happens at three layers, and interviewers probe all three:

1. **Bundle/network layer** — how much JavaScript ships, when it ships, and how images/fonts load. Levers: tree-shaking, lazy loading, `@defer`, `NgOptimizedImage`, differential builds.
2. **Runtime/rendering layer** — how often and how expensively change detection runs. Levers: `OnPush`, immutability, `trackBy` / `track`, pure pipes, virtual scrolling.
3. **Measurement layer** — proving the above actually worked. Levers: Angular DevTools profiler, Chrome Performance tab, Lighthouse.

A senior candidate is expected to reason about the *mechanism* behind each optimization (why does `OnPush` skip a subtree? why does `@defer` produce a separate chunk?), not just recite "use `OnPush` for performance." This chapter goes mechanism-first.

2. Core Concepts

2.1 Bundle Size Optimization

Angular's production build (`ng build` with `optimization: true`, the default for `production` configuration) runs:

- **Ahead-of-Time (AOT) compilation** — templates compiled to JS at build time, no template compiler shipped to the browser.
- **Tree-shaking** via Terser/esbuild, relying on ES modules' static `import` / `export` structure.
- **Minification & mangling** of identifiers.
- **Scope hoisting** to reduce module wrapper overhead.
- **Differential loading is gone** in modern Angular (CLI now targets evergreen browsers by default) — but budgets in `angular.json` (`budgets: [{type: "initial", maximumWarning, maximumError}]`) still gate bundle growth in CI.

Practical levers a developer controls:

- Avoid barrel-file imports that pull in an entire library when you need one function (`import { debounce } from 'lodash'` instead of `import _ from 'lodash'`).
- Prefer native APIs over utility libraries where trivial (`structuredClone` vs `Lodash.cloneDeep`).
- Use `sideEffects: false` in `package.json` for libraries you author, so bundlers can drop unused exports safely.
- Lazy-load routes and heavy widgets instead of bundling everything into `main.js`.
- Use esbuild-based builder (`@angular-devkit/build-angular:application`, default since Angular 17) for faster builds and smaller output vs the legacy Webpack builder.

2.2 Tree-Shaking — Mechanics

Tree-shaking removes code that is provably unreachable from the entry point. It relies on:

- **Static ES module syntax** (`import / export`, not `require / module.exports`) so the bundler can build a static dependency graph without executing code.
- **Purity annotations** (`/*#__PURE__*/`) on function calls with no side effects, telling the minifier "this call's result is unused $\Rightarrow$ safe to delete," which is why Angular decorators compile to annotated helper calls.
- `providedIn: 'root'` on injectables — this is itself a tree-shaking mechanism. A service registered via `providedIn: 'root'` is only included in the bundle if something actually injects it. Registering it in an `NgModule`'s `providers` array instead defeats this — it's included unconditionally once that module is reachable.
- **Standalone components** (default since Angular 17-19) improve tree-shaking further because there's no `NgModule` graph forcing eager inclusion of declared components — the compiler can trace the actual component/directive/pipe usage graph from each entry point.

Tree-shaking cannot remove code with detectable side effects at the module top level (e.g., `console.log()` at import time, or polyfill libraries that patch globals) unless the package explicitly marks itself side-effect-free.

2.3 Lazy Loading Strategy

- **Route-level lazy loading:** `loadComponent: () => import('./admin/admin.component').then(m => m.AdminComponent)` or `loadChildren` for child route groups. Each becomes a separate chunk fetched only on navigation.
- **Preloading strategies:** `PreloadAllModules` (Angular preloads every lazy route in the background after the app stabilizes) vs a **custom `PreloadingStrategy`** that reads route `data` flags to selectively preload only routes likely to be visited (e.g., based on user role or a "preload: true" flag), balancing initial-load cost against navigation latency.
- **Component-level lazy loading** via `@defer` (Angular 17+) — finer grained than routes; can lazy-load a component that isn't behind a route at all (e.g., a comments section below the fold).
- **Library lazy loading** — dynamic `import()` for a heavy library (chart engine, PDF renderer) only when the feature using it is invoked, independent of routing.

Strategy selection in interviews: route-based lazy loading answers "reduce initial bundle," `@defer` answers "reduce initial bundle *and* defer non-critical rendering work even within an already-loaded route."

2.4 OnPush + Immutability

Default (`ChangeDetectionStrategy.Default`) change detection walks the **entire component tree** top-to-bottom on every trigger (DOM event, HTTP response resolving inside `Zone.js`, timer, etc.), checking every binding and running `ngDoCheck` in every component, regardless of whether that component's data changed.

`ChangeDetectionStrategy.OnPush` tells Angular a component's subtree only needs checking when one of these happens:

1. An `@Input()` reference changes (checked by `===`, not deep equality).
2. An event originates from within the component or its template (a `click` etc. inside its own view).

3. An `Observable` bound with `| async` emits.
4. Change detection is explicitly triggered via `ChangeDetectorRef.markForCheck()` (or `detectChanges()`).

This is why **immutability matters**: if you mutate an array/object in place (`this.items.push(x)`) and pass the same reference down, the `===` check on the `@Input` sees no change and Angular skips that subtree — the view silently goes stale. Correct pattern: `this.items = [...this.items, x]`, producing a new reference so `onPush` detects the change.

With signals (Angular 17+), this problem mostly disappears: signal reads register fine-grained reactive dependencies, and updates propagate regardless of reference equality semantics on ordinary bindings, because the signal itself is the identity being tracked, not the value it wraps.

2.5 trackBy / track

`*ngFor` (or the new `@for` control-flow syntax) without a tracking function makes Angular diff lists by **index/object identity** by default — if the array reference changes (e.g., after a filter or an immutable update), Angular treats every item as new: it destroys and recreates every DOM node and every child component instance for that list, even if the underlying data objects are unchanged. This is expensive for large lists and also destroys component state (form focus, animations, scroll position) inside list items.

- **Legacy syntax:** `*ngFor="let item of items; trackBy: trackById"` with `trackById(index: number, item: Item) { return item.id; }`.
- **New control flow (Angular 17+):** `@for (item of items; track item.id) { ... }` — `track` is mandatory in the new syntax (you must supply an expression; there's no silent default-to-identity fallback), which is a deliberate API design push toward always tracking correctly. `track $index` is allowed but only appropriate when items never reorder/insert/delete except at the end.

Mechanically, `trackBy / track` lets Angular's list-diffing algorithm map old DOM nodes to new data by a stable key instead of by array position, so only inserted/removed/moved items cause DOM churn — updated-in-place items get their bindings refreshed without node recreation.

2.6 Pure Pipes over Method Calls

Calling a method directly in a template — `{{ getFullName(user) }}` — re-executes that method **on every change detection run**, regardless of whether `user` changed, because Angular has no way to know if the method is deterministic or side-effect-free.

A **pure pipe** (`@Pipe({name: 'fullName', pure: true})`, the default) is only re-invoked when its **input reference** changes (primitives by value, objects/arrays by `===`). Angular memoizes the last input/output pair internally per pipe instance in the template. This converts an $O(n)$ recompute-every-CD-cycle into effectively $O(1)$ most cycles.

Caveat: pure pipes suffer the *same* mutation trap as `onPush` — mutating an array in place won't re-trigger a pure pipe bound to that array, because the reference didn't change. **impure** pipes (`pure: false`) re-run every CD cycle regardless (useful for things like a genuinely mutable in-place-sorted array, but expensive — effectively opts back into the method-call cost, so use sparingly and prefer fixing the immutability instead).

2.7 Virtual Scrolling

`@angular/cdk/scrolling`'s `<cdk-virtual-scroll-viewport>` renders only the DOM nodes currently visible in the viewport (plus a small buffer), recycling nodes as the user scrolls, rather than materializing thousands of DOM nodes for a large list. This bounds DOM size (and CD cost, since CD only walks *rendered* nodes) independent of list length.

```
<cdk-virtual-scroll-viewport itemSize="48" class="viewport">
  <div *cdkVirtualFor="let item of items; trackBy: trackById">{{ item.name }}</div>
</cdk-virtual-scroll-viewport>
```

`itemSize` (fixed-size strategy) lets the CDK precompute total scrollable height without measuring every item. For variable-height items, a custom `VirtualScrollStrategy` is needed since the default `FixedSizeVirtualScrollStrategy` assumes uniform height.

2.8 Image Optimization (`NgOptimizedImage`)

The `NgOptimizedImage` directive (`ngSrc` instead of `src`, from `@angular/common`) enforces/automates image best practices:

- Requires explicit `width/height` (or `fill`) to prevent Cumulative Layout Shift (CLS) by reserving space before the image loads.
- Automatically adds `fetchpriority="high"` for images marked `priority` (typically the LCP — Largest Contentful Paint — image), and warns in dev mode if the LCP image is *not* marked priority.
- Generates a `srcset` automatically when using a configured image loader (e.g., Cloudinary, Imgix, or a custom loader), enabling responsive images without hand-writing `srcset` strings.
- Lazy-loads (`loading="lazy"`) all non-priority images by default.
- Warns at dev time about missing dimensions, oversized images relative to their rendered size, and other common mistakes — this is a compile/runtime **linting** feature as much as a rendering optimization.

2.9 Deferrable Views (`@defer`)

`@defer` (stable Angular 17+) splits part of a template into a **separate lazy-loaded JS chunk** at build time, deferring both the fetching and the rendering of that content until a trigger condition fires.

Triggers:

- `on viewport` — when the block enters the viewport (uses `IntersectionObserver`).
- `on idle` — when the browser is idle (`requestIdleCallback`), the default trigger if none specified.
- `on interaction` — on click/keydown on the block or a referenced trigger element.
- `on hover` — mouseenter/focus.
- `on timer(xms)` — after a fixed delay.
- `on immediate` — as soon as possible after the initial render (still async, just no waiting condition).
- `when <expression>` — a custom boolean condition, re-evaluated on every CD cycle until true.

Sub-blocks:

- `@placeholder (minimum 500ms) { }` — shown before the trigger fires.
- `@loading (minimum 1s; after 100ms) { }` — shown while the chunk is being fetched (`after` avoids flashing a spinner for near-instant loads; `minimum` avoids flicker if it resolves too fast).
- `@error { }` — shown if the dynamic import fails.

Only standalone components/directives/pipes referenced *exclusively* inside the `@defer` block get split into the deferred chunk — if the same component is also used elsewhere eagerly in the same file, it can't be deferred (Angular emits a diagnostic).

2.10 Avoiding Unnecessary Re-renders

Consolidated checklist, tying the above together:

- Use `onPush` everywhere feasible; treat `Default` strategy as the exception, not the norm.
- Never mutate `@Input`-bound objects/arrays.
- Move template expressions with computation into pure pipes or memoized signals (`computed()`), not inline method calls.
- Always provide `track / trackBy` on loops.
- Detach `ChangeDetectorRef` (`detach()`) for components that render fully static/one-time content and re-attach only if needed.
- Run non-UI-affecting work (analytics timers, socket keep-alives) `NgZone.runOutsideAngular()` so it doesn't schedule CD ticks at all.
- Prefer signals for state that changes frequently in isolated widgets — signal-based reactivity (Angular's zoneless roadmap) only re-renders the components that actually *read* the changed signal, versus Zone.js-triggered CD which (absent `onPush`) walks the whole tree.

2.11 Profiling Tools

- **Angular DevTools** (browser extension): a "Profiler" tab that records a timeline of change-detection cycles, showing per-component render duration and *why* a component was checked. Useful for spotting components checked every cycle that shouldn't be (missing `onPush`), or components with heavy synchronous render logic.
- **Chrome Performance tab**: general-purpose JS profiling — flame charts showing scripting/rendering/painting time, long tasks (>50ms, which block input responsiveness and hurt Interaction to Next Paint), and can be correlated with Angular's `NgZone` `onUnstable` / `onStable` markers to see how long a CD cycle actually took at the browser level.
- **Lighthouse**: audits Core Web Vitals (LCP, CLS, INP/FID, TBT) end-to-end and flags render-blocking resources, unoptimized images (surfacing exactly the issues `NgOptimizedImage` targets), unused JS (surfacing tree-shaking/lazy-loading gaps), and excessive main-thread work.

3. Code Examples

3.1 `@defer` with all sub-blocks and multiple trigger types

```
import { Component, signal } from '@angular/core';
```

```

import { CommentsComponent } from './comments/comments.component';
import { HeavyChartComponent } from './heavy-chart/heavy-chart.component';

@Component({
  selector: 'app-article',
  standalone: true,
  imports: [CommentsComponent], // HeavyChartComponent is NOT imported eagerly
  template: `
    <article>
      <h1>{{ title() }}</h1>
      <p>{{ body() }}</p>

      <!-- Load the chart only when it scrolls into view -->
      @defer (on viewport; prefetch on idle) {
        <app-heavy-chart [data]="chartData()" />
      } @placeholder (minimum 200ms) {
        <div class="chart-placeholder">Chart will appear when scrolled into view</div>
      } @loading (after 150ms; minimum 500ms) {
        <app-spinner />
      } @error {
        <p class="error">Could not load chart. <button (click)="retry()">Retry</button></p>
      }

      <!-- Load comments only after the user explicitly asks -->
      <button #showComments (click)="null">Show comments</button>
      @defer (on interaction(showComments)) {
        <app-comments [articleId]="articleId()" />
      } @placeholder {
        <p>Comments hidden – click above to load</p>
      }
    </article>
  `,
})
export class ArticleComponent {
  title = signal('Angular Performance Deep Dive');
  body = signal('...');
  chartData = signal<number[]>([]);
  articleId = signal(42);

  retry() {
    // re-triggering is handled by re-evaluating the `on viewport` condition
    // after clearing any cached failure state, or by reloading chartData.
  }
}

```

`HeavyChartComponent` and `CommentsComponent`'s implementation modules are compiled into **separate lazy chunks** because they're referenced only inside `@defer` blocks and not in the component's `imports` array used for eager rendering.

3.2 NgOptimizedImage usage

```

import { Component } from '@angular/core';

```

```

import { NgOptimizedImage } from '@angular/common';

@Component({
  selector: 'app-hero-banner',
  standalone: true,
  imports: [NgOptimizedImage],
  template: `
    <!-- LCP image: eagerly loaded, high fetch priority -->
    <img
      ngSrc="hero/banner.jpg"
      width="1200"
      height="600"
      priority
      alt="Product banner"
    />

    <!-- Below-the-fold image: lazy-loaded automatically -->
    <img
      ngSrc="assets/thumbnails/product-42.jpg"
      width="300"
      height="200"
      alt="Product thumbnail"
    />

    <!-- Responsive fill container (parent must be position: relative) -->
    <div class="avatar-wrapper">
      <img ngSrc="assets/avatar.jpg" fill alt="User avatar" />
    </div>
  `,
})
export class HeroBannerComponent {}

```

```

// app.config.ts – registering a third-party image loader for automatic srcset generation
import { ApplicationConfig } from '@angular/core';
import { provideImgixLoader } from '@angular/common';

export const appConfig: ApplicationConfig = {
  providers: [
    provideImgixLoader('https://my-app.imgix.net'),
  ],
};

```

3.3 OnPush + trackBy/track combo, immutable updates

```

import { ChangeDetectionStrategy, Component, signal } from '@angular/core';

interface Todo { id: number; text: string; done: boolean; }

@Component({
  selector: 'app-todo-list',
  standalone: true,

```

```

changeDetection: ChangeDetectionStrategy.OnPush,
template: `
  <ul>
    @for (todo of todos(); track todo.id) {
      <li [class.done]="todo.done">
        {{ todo.text }}
        <button (click)="toggle(todo.id)">Toggle</button>
      </li>
    }
  </ul>
  <button (click)="addTodo('New task')">Add</button>
`,
})
export class TodoListComponent {
  todos = signal<Todo[]>([
    { id: 1, text: 'Learn OnPush', done: false },
    { id: 2, text: 'Learn trackBy', done: false },
  ]);

  toggle(id: number) {
    // Immutable update: new array, new object for the changed item.
    // Required for OnPush to see a new @Input reference if todos()
    // were passed down as an @Input to a child; here, since it's a
    // signal read directly in this OnPush component's own template,
    // the signal's change tracking handles it regardless – but the
    // immutable pattern remains correct practice for interop with
    // plain @Input-based children.
    this.todos.update(list =>
      list.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }

  addTodo(text: string) {
    this.todos.update(list => [
      ...list,
      { id: Math.max(0, ...list.map(t => t.id)) + 1, text, done: false },
    ]);
  }
}

```

Legacy (`*ngFor` + `trackBy` + manual `ChangeDetectorRef`) equivalent for contrast:

```

@Component({
  selector: 'app-todo-list-legacy',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <li *ngFor="let todo of todos; trackBy: trackById">{{ todo.text }}</li>
  `,
})
export class TodoListLegacyComponent {
  @Input() todos: Todo[] = [];

  trackById(index: number, todo: Todo): number {

```



```
    return todo.id;
  }
}
```

4. Internal Working

4.1 How `@defer` splits and lazy-loads

At compile time, the Angular compiler identifies each `@defer` block's template content and any standalone dependencies (components/directives/pipes) used *only* within it. It emits:

- A **dynamic** `import()` call for those dependencies, which bundlers (esbuild/Webpack/Vite) recognize as a code-splitting point, producing a separate output chunk (e.g., `chunk-xyz.js`) not included in the initial bundle.
- Runtime instructions (`ɵdefer`, `ɵdeferOnIdle`, `ɵdeferOnViewport`, etc.) embedded in the compiled component that register the trigger condition (an `IntersectionObserver`, `requestIdleCallback`, event listener, or timer) at runtime.
- When the trigger fires, Angular calls the dynamic `import()`, which the browser fetches over the network (a real HTTP request, visible in DevTools' Network tab as a separate chunk request); once resolved, Angular instantiates the component(s) inside the deferred block and swaps the `@placeholder / @loading` view for the real content.
- `prefetch on idle` (or any prefetch trigger) allows fetching the chunk *ahead of* render-trigger time — e.g., fetch as soon as idle, but only render `on viewport` — decoupling network fetch timing from render timing for a smoother perceived experience.

4.2 How the build pipeline tree-shakes

1. **TypeScript** → **AOT template compilation** produces plain ES module JS/TS with static imports (Ivy instructions like `ɵdefineComponent`).
2. The bundler (esbuild in the modern `application` builder) constructs a **module graph** from the entry point(s) by statically analyzing `import / export` statements — this requires ESM, since CommonJS's dynamic `require()` calls can't be statically resolved.
3. **Mark phase**: starting from used exports at the entry point, the bundler marks every export transitively reachable.
4. **Sweep phase**: unmarked exports (and, transitively, code only reachable through them) are eliminated from output, provided the bundler can prove no side effects are lost — governed by `sideEffects` in `package.json` and `/*#__PURE__*/` annotations on constructor/function calls.
5. **Minification pass** (Terser/esbuild minifier) then does dead-code elimination on what's left (unreachable branches, unused local variables), and mangles/shortens identifiers.
6. Angular-specific: DI tokens registered with `providedIn: 'root'` compile to a factory function only referenced if actually injected somewhere in the reachable graph — so an unused `@Injectable({providedIn: 'root'})` service is fully removed, whereas one in an `NgModule.providers` array is retained as long as that module is loaded, regardless of whether anything injects it.

4.3 How DevTools profiles CD cycles

Angular instruments each `ApplicationRef.tick()` (the entry point for a change detection pass, invoked by Zone.js when it detects the zone going "stable" after async work, or by manual `ChangeDetectorRef / ApplicationRef.tick()` calls). Angular DevTools' Profiler:

- Hooks into these tick boundaries and records, for each component instance visited, the time spent in that component's `detectChanges` — this includes bindings evaluation and lifecycle hooks like `ngDoCheck / ngOnChanges`.
- Renders a **bar-chart timeline** of ticks (each tick is one bar/frame), and clicking a tick shows a **flame-graph-like breakdown** per component, letting you see both "how long did this cycle take" and "which components were checked in it."
- Distinguishes components skipped due to `OnPush` (reference-unchanged inputs) from those actually re-rendered, which is the direct way to verify an `OnPush` optimization is working rather than assuming it from reading the code.
- Uses the same instrumentation Angular exposes for the (now largely superseded by DevTools) `ng.profiler.timeChangeDetection()` console API historically used for manual profiling.

The Chrome Performance tab sits one level below this: it captures actual browser main-thread activity (Scripting/Rendering/Painting/System categories), so an Angular CD cycle shows up as a JS call stack rooted around `Zone.prototype.runTask` → `ApplicationRef.tick` → recursive `View.detectChangesInternal` calls, letting you correlate Angular-level ticks with browser-level long tasks, layout thrashing, and paint costs that Angular DevTools alone doesn't show.

5. Edge Cases & Gotchas

- **Mutating input arrays under `OnPush`:** `this.items.sort(...)` or `.push(...)` on an `@Input`-bound array won't trigger re-render in a child `OnPush` component even though the data changed, because the reference is identical. This is the single most common "why isn't my UI updating" bug once teams adopt `OnPush`.
- **async pipe + `OnPush` interaction:** the `async` pipe internally calls `markForCheck()` on emission, which is *why* `OnPush` components correctly update when bound to an `Observable` via `| async` even without manual `ChangeDetectorRef` calls — but only for the specific view, not proactively subscribing/propagating information about deep object mutations inside emitted objects.
- **`track $index in @for`:** acceptable only for append-only, never-reordered lists. Using it on a sortable/filterable list causes Angular to match old DOM nodes to new data by position, producing the very identity-confusion bugs (wrong item retains focus/animation state) that `track` was introduced to prevent — this is worse than not thinking about it because it looks correct until reordering happens.
- **Deferring a component used elsewhere eagerly:** `@defer` requires the deferred dependencies to be standalone and *not* also referenced outside any `@defer` block in the same component; otherwise Angular can't safely split them into a separate chunk and raises a compile-time error.
- **`@defer` with SSR:** on the server, deferred blocks render their placeholder (or, with incremental hydration configured, can be included fully) — naive assumptions that `@defer` content is present in server-rendered HTML for SEO purposes are wrong unless explicitly configured for eager SSR rendering via hydration boundaries.

- **NgOptimizedImage without configured loader:** `ngSrc` alone (no loader provider) does *not* generate a `srcset` — you get lazy-loading/priority/CLS-prevention benefits but not responsive images, a commonly missed distinction.
- **Impure pipes silently reintroducing the method-call cost:** marking a pipe `pure: false` to "fix" a stale-pipe-on-mutated-array bug re-runs it every CD cycle — trading a correctness bug for a performance regression instead of fixing the root cause (mutation).
- **Virtual scroll + dynamic item height:** the default `cdk-virtual-scroll-viewport` assumes fixed `itemSize`; variable-height content (e.g., chat messages, expandable rows) requires a custom `VirtualScrollStrategy`, and using the default with variable heights causes visual jumping/incorrect scrollbar sizing.
- **Preloading strategy over-preloading:** `PreloadAllModules` eagerly fetches every lazy chunk shortly after boot — on a large app with many rarely visited admin routes, this can silently reintroduce the bundle-size problem lazy loading was meant to solve, just delayed slightly; a custom strategy filtering by route `data.preload` flag avoids this.
- **providedIn: 'root' vs module providers and tree-shaking:** a service registered only in a lazy-loaded feature module's `providers` array is *still* eagerly bundled with that module, but a forgotten `providedIn: 'root'` on something meant to be tree-shaken away (e.g., feature-flagged debug tooling) will ship in the main bundle even if never injected, if it's still imported and referenced anywhere in eagerly-loaded code, because it's the reachability, not the annotation alone, that matters.

6. Interview Questions & Answers

Q1. What's the difference between `ChangeDetectionStrategy.Default` and `OnPush`?

A: `Default` checks a component (and all its bindings/lifecycle hooks) on every change-detection cycle regardless of whether its data changed. `OnPush` skips a component's subtree unless one of: an `@Input` reference changed (`===` check), an event originated within that component's own template, an `async`-pipable observable emitted, or `markForCheck()` / `detectChanges()` was called explicitly.

Q2. Why doesn't mutating an array in place trigger an `OnPush` child to re-render?

A: `OnPush` compares `@Input` values by reference identity, not deep equality. Mutating (`push`, `sort`, property assignment) keeps the same array/object reference, so the `===` check reports "unchanged" and Angular skips checking that component, even though the underlying data differs. The fix is to always produce a new reference on update (`spread`/`map`/`filter`, or `structuredClone`).

Interviewer intent: This question filters candidates who've only memorized "use `OnPush` for performance" from those who understand the mutation trap it introduces — a very common real-world bug source.

Q3. What does `trackBy` (or `track` in `@for`) actually change about rendering?

A: Without it, Angular's list-diffing defaults to identity/positional comparison of the *array reference and items*; when the array reference changes (e.g. after an immutable update), every item is treated as new, so Angular destroys and recreates the DOM node (and any child component instance, losing its state) for every list item. `trackBy` / `track` supplies a stable key (e.g., `item.id`) so Angular can match old nodes to new data, updating bindings in place and only creating/destroying/moving nodes for items that were actually added/removed/reordered.

Q4. Why is `track` mandatory in the new `@for` syntax but optional (and often omitted) in `*ngFor`?

A: `*ngFor` defaults to positional identity tracking when no `trackBy` is given, which is easy to forget and a frequent silent-performance-bug source. The Angular team made `track` a required part of `@for`'s syntax specifically to force developers to make a conscious choice, eliminating the "forgot `trackBy`" class of bug entirely.

Q5. How does a pure pipe differ from calling a method in a template, performance-wise?

A: A method call in a template re-executes on every CD cycle unconditionally, since Angular can't know if it's side-effect-free. A pure pipe is only re-invoked when its input reference(s) change — Angular memoizes the last input/output per pipe instance — turning a per-cycle recomputation into a no-op most cycles. The tradeoff: pure pipes are also subject to the reference-equality trap (won't detect in-place mutation).

Q6. Walk through what happens, chunk-wise, when you write `@defer (on viewport) { <app-heavy/> }`.

A: At compile time, if `HeavyComponent` (standalone) is used only inside this `@defer` block, the Angular compiler emits a dynamic `import()` for it, which the bundler recognizes as a split point and outputs as a separate chunk not included in the initial bundle. At runtime, Angular registers an `IntersectionObserver` on the placeholder's position; when it scrolls into view, Angular triggers the dynamic import (an actual network fetch), and once resolved, instantiates `HeavyComponent` and replaces the placeholder view with it.

Q7. What's the difference between `@placeholder`, `@loading`, and `@error` in `@defer`?

A: `@placeholder` shows before the trigger condition has fired at all. `@loading` shows once the trigger fired and the chunk is being fetched (supports `after` to avoid flashing on fast loads, and `minimum` to avoid flicker on very fast ones). `@error` shows if the dynamic import itself fails (e.g., network failure) — a case that route-level lazy loading doesn't give template-level control over.

Q8. How would you decide between route-level lazy loading and `@defer` for a given piece of UI?

A: Route-level lazy loading is the right tool when the content maps to actual navigation — a distinct page/section the user goes "into." `@defer` is the right tool for content that's part of an already-loaded page but expensive or non-critical to render immediately — below-the-fold widgets, modals, rarely-used admin panels embedded in a dashboard, heavy third-party-library-backed components (charts, rich text editors) that shouldn't block the initial page's interactivity.

Interviewer intent: Checks whether the candidate sees these as complementary tools operating at different granularities, rather than treating "lazy loading" as one undifferentiated concept.

Q9. What is `providedIn: 'root'` and how does it interact with tree-shaking?

A: It registers an injectable at the root injector via a tree-shakeable factory function, rather than listing it in an `NgModule`'s `providers` array. If nothing in the app ever injects that service, the bundler can prove it's unreachable and remove it entirely from output. Providing it in a module's `providers` array instead makes it bundled unconditionally whenever that module is loaded, regardless of actual usage — defeating tree-shaking for that provider.

Q10. Why does virtual scrolling reduce both DOM size and change-detection cost?

A: `cdk-virtual-scroll-viewport` only instantiates and renders DOM nodes for items currently in (or near) the viewport, recycling them as the user scrolls, instead of materializing every item in the list. Since Angular's change detection only walks *rendered* view nodes, bounding the rendered set to, say, 20 visible rows instead of 10,000 total rows bounds CD cost per cycle to a constant independent of list size, in addition to the obvious DOM-node-count/memory savings.

Q11. What specific problems does `NgOptimizedImage` solve that a plain `<img>` doesn't?

A: It enforces explicit `width/height` (or `fill`) to reserve layout space and prevent CLS; auto-applies `loading="lazy"` to non-priority images and `fetchpriority="high"` + eager loading to the `priority`-marked (typically LCP) image; can auto-generate responsive `srcset`s via a configured image CDN loader; and emits dev-time warnings for common mistakes (missing dimensions, LCP image not marked `priority`, oversized source images). A plain `<img>` gives you none of this automatically or with the same in-dev diagnostics.

Q12. How would you use Angular DevTools' profiler to diagnose a component that renders too often?

A: Open the Profiler tab, record an interaction, then inspect the recorded ticks (bars on the timeline). Click into a tick to see the flame-graph-style breakdown of which components were checked and how long each took. A component missing `OnPush` (or one whose inputs are being mutated rather than replaced) will show up as checked on every tick regardless of whether its actual displayed data changed — the fix is confirmed by re-profiling after adding `OnPush` /fixing immutability and seeing that component drop out of ticks where its inputs didn't change.

Q13. Chrome Performance tab vs Angular DevTools profiler — when would you reach for each?

A: Angular DevTools gives Angular-semantic information — which *components* were checked, in which CD cycle, and (with `OnPush`) why one was skipped — but nothing below that abstraction. The Chrome Performance tab gives raw browser main-thread activity — actual JS call stacks, long tasks (>50ms), layout/paint/composite costs, and can reveal problems Angular DevTools can't, like a synchronous function inside a component doing expensive DOM measurement (forced reflow) or a third-party script blocking the main thread. Use Angular DevTools first to find *which component* is problematic, then Chrome Performance to see *what it's actually doing* at the browser level during its render.

Interviewer intent: Tests whether the candidate treats these as a layered diagnostic workflow (Angular-level → browser-level) rather than interchangeable/redundant tools.

Q14. Why might `PreloadAllModules` hurt performance on a large application?

A: It fetches every lazy-loaded route's chunk shortly after the app becomes stable, regardless of whether the user is likely to visit that route. On an app with many large, rarely-used routes (e.g., admin-only sections), this reintroduces much of the network cost lazy loading was meant to avoid — just deferred by a few seconds instead of eliminated — and can compete for bandwidth/CPU with the user's actual current task. A custom `PreloadingStrategy` that reads route `data` (e.g., `{ preload: true }` only on routes worth prefetching, informed by real usage analytics) is the more deliberate alternative.

Q15. Can `@defer` blocks reference components that are also used eagerly elsewhere in the same component?

A: No — if a standalone dependency is imported/used both inside a `@defer` block and outside it (eagerly) within the same component's template, Angular can't safely split it into a separate lazy chunk (it would already need to be in the eager bundle), and the compiler raises a diagnostic/error. The dependency must be exclusively deferred-block-only to be split out.

7. Quick Revision Cheat Sheet

- **Tree-shaking** needs ESM (`import / export`), works via mark-and-sweep from entry points; `providedIn: 'root'` is tree-shakeable, module `providers` arrays are not.
- **Lazy loading:** routes (`loadComponent / loadChildren`) for navigation-level splitting; `PreloadingStrategy` (custom > `PreloadAllModules` for large apps) to balance eager vs deferred fetch.

- `@defer`: splits template into its own chunk at compile time; triggers — `on viewport | idle | interaction | hover | timer(ms) | immediate | when <expr>`; sub-blocks — `@placeholder(min)`, `@loading(after; min)`, `@error`; `prefetch` decouples fetch timing from render timing.
- `OnPush`: skips subtree unless `@Input` reference changes, event originates inside, `async` pipe emits, or `markForCheck()` / `detectChanges()` called. Always pair with **immutable updates**.
- `trackBy` / `track`: prevents full DOM/component recreation on list re-render; `track` is mandatory in `@for`; never use `track $index` on reorderable lists.
- **Pure pipes**: memoized by input reference, cheaper than template method calls; same mutation-blindness as `OnPush`. Avoid `pure: false` as a mutation-bug workaround.
- **Virtual scrolling** (`cdk-virtual-scroll-viewport`): bounds rendered DOM + CD cost to viewport size, independent of list length; fixed `itemSize` strategy by default, custom strategy needed for variable heights.
- `NgOptimizedImage`: `ngSrc` + required `width/height` (or `fill`); `priority` for LCP image (eager + high fetch priority); auto lazy-loads others; needs a configured loader for auto `srcset`.
- **Profiling layers**: Angular DevTools (component-level CD ticks) → Chrome Performance (browser-level flame charts/long tasks) → Lighthouse (Core Web Vitals audit, end-to-end).
- **Golden rule tying it together**: reduce what ships (tree-shaking + lazy loading + `@defer`), reduce how often it re-renders (`OnPush` + immutability + `trackBy`/pure pipes), reduce what's on-screen at once (virtual scroll), then measure to confirm (DevTools/Performance/Lighthouse) rather than assume.

Created By - Durgesh Singh